S.O.L.I.D.

principles

S. Single Responsibility Principle

O. Open Closed Principle

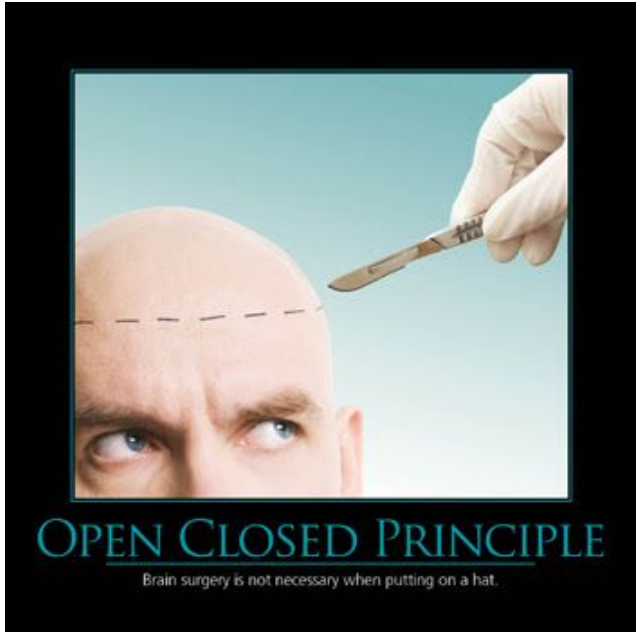
L. Liskov Substitution Principle

I. Interface Segregation Principle

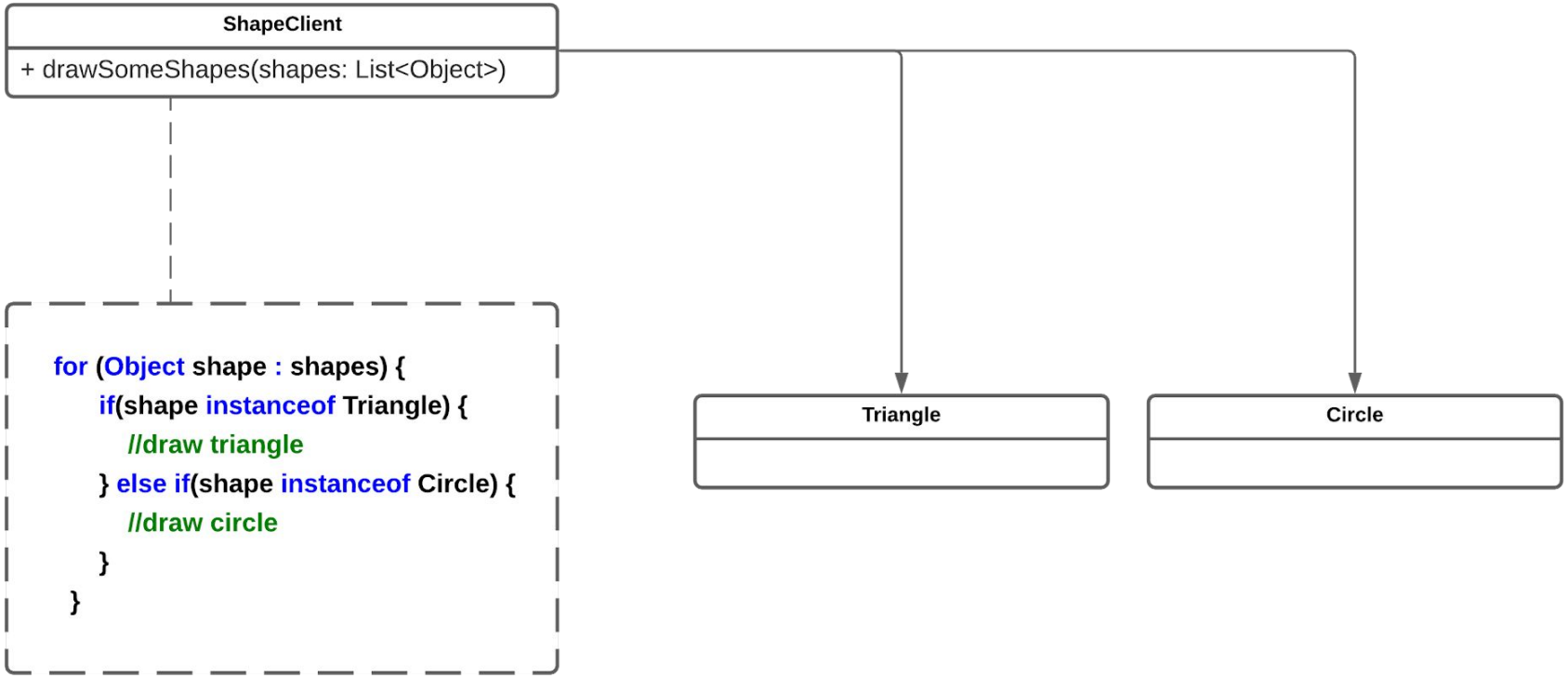
D. Dependency Inversion Principle

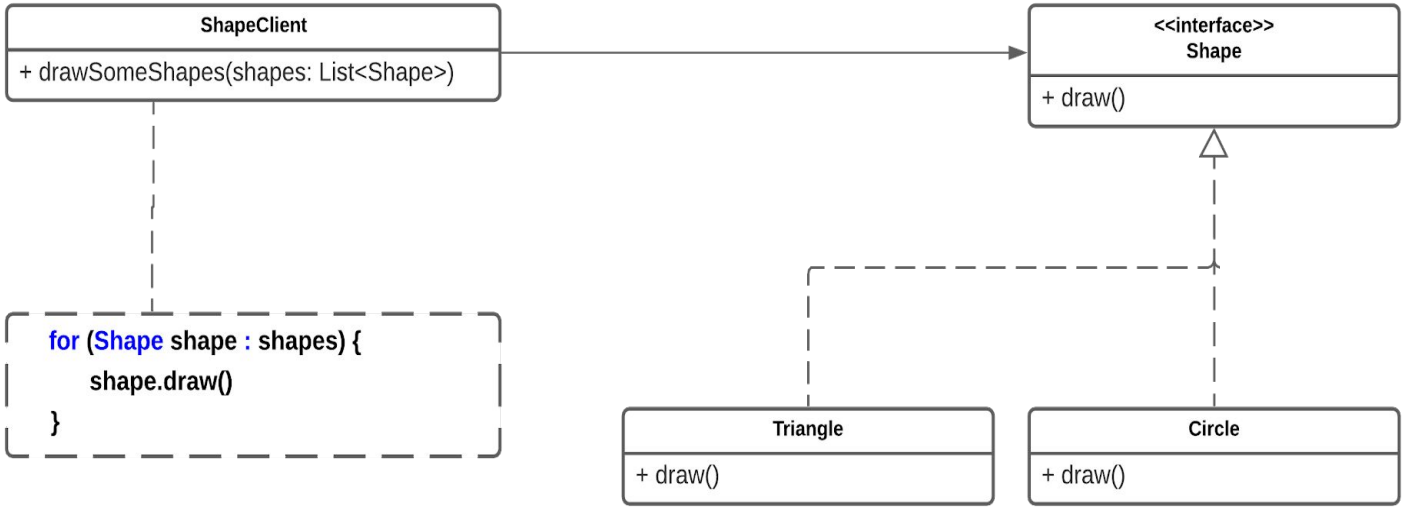
OPEN-CLOSED PRINCIPLE

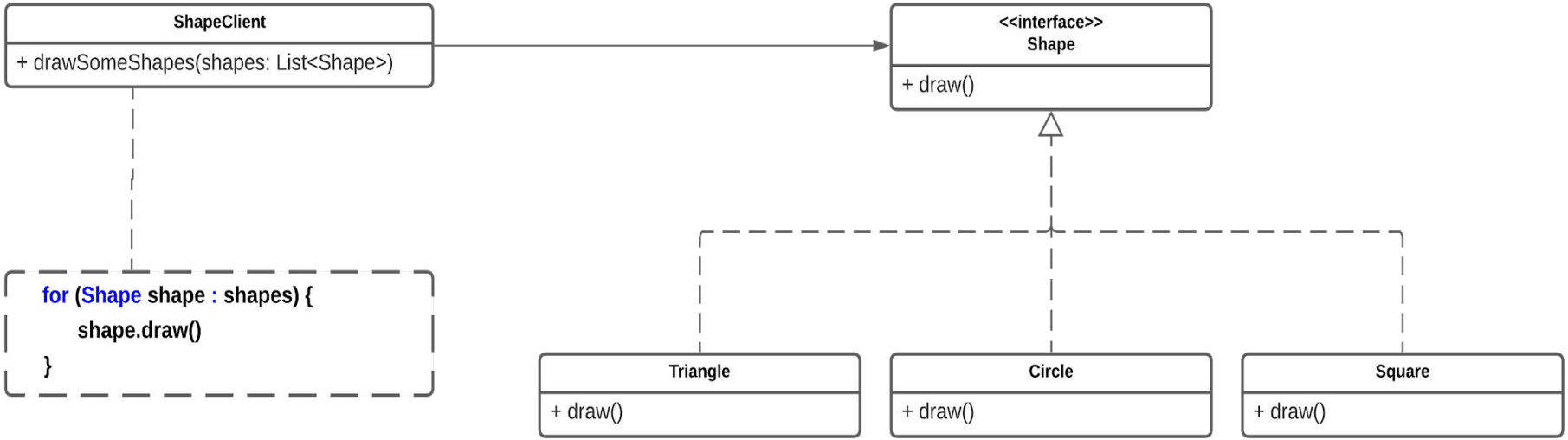
A class should be open for extension, but closed for modification!



The shapes example


ShapeClient
+ drawSomeShapes(shapes: List<Object>)
for (Object shape : shapes) {
if(shape instanceof Triangle) {
//draw triangle
} else if(shape instanceof Circle) {
//draw circle
}
}
Triangle
Circle

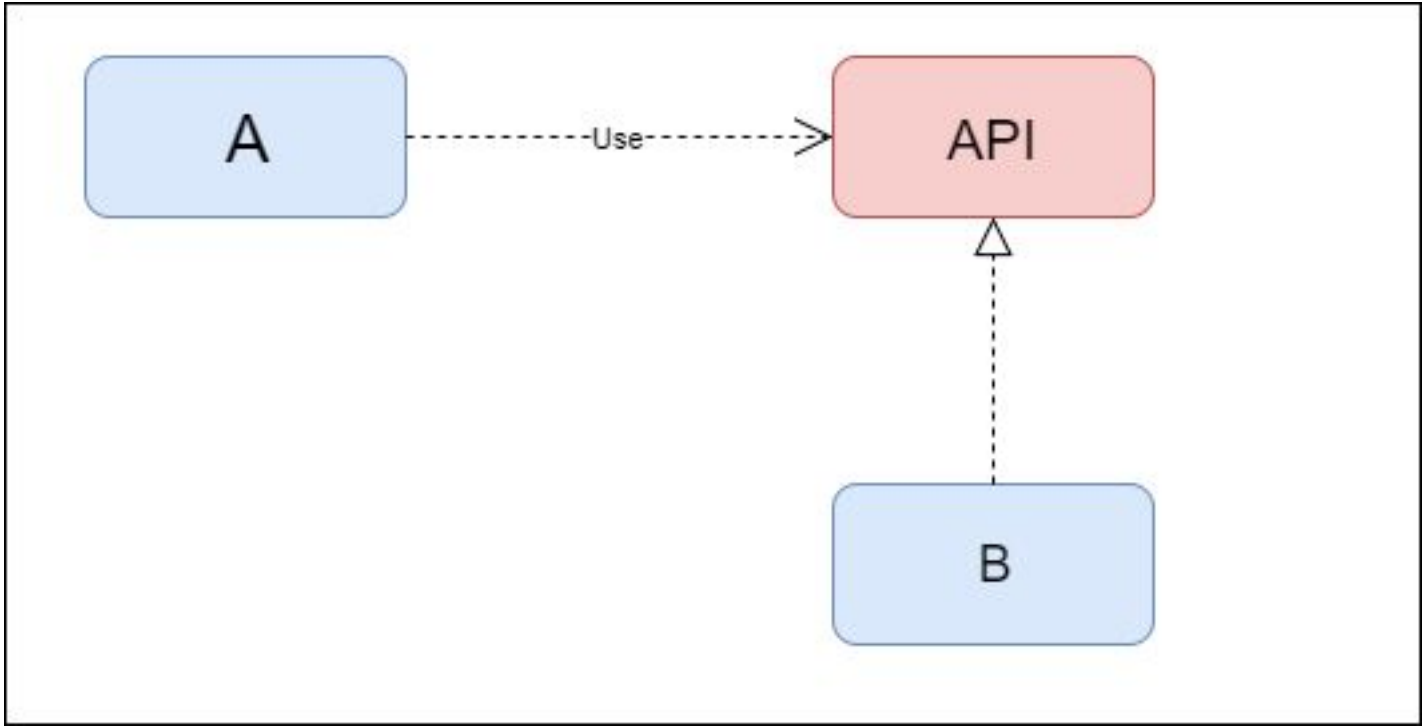

ShapeClient
+ drawSomeShapes(shapes: List<Shape>)
<<interface>>
Shape
+ draw()
for (Shape shape : shapes) {
shape.draw()
}
Triangle
+ draw()
Circle
+ draw()


ShapeClient
+ drawSomeShapes(shapes: List<Shape>)
<<interface>>
Shape
+ draw()
for (Shape shape : shapes) {
shape.draw()
}
Triangle
+ draw()
Circle
+ draw()
Square
+ draw()

Open-closed

- No Runtime type checks
  - No instanceof
  - No boolean flags
  - No switch statements
- Reuse
- Add functionality without modifying the existing code
- Strategic closure

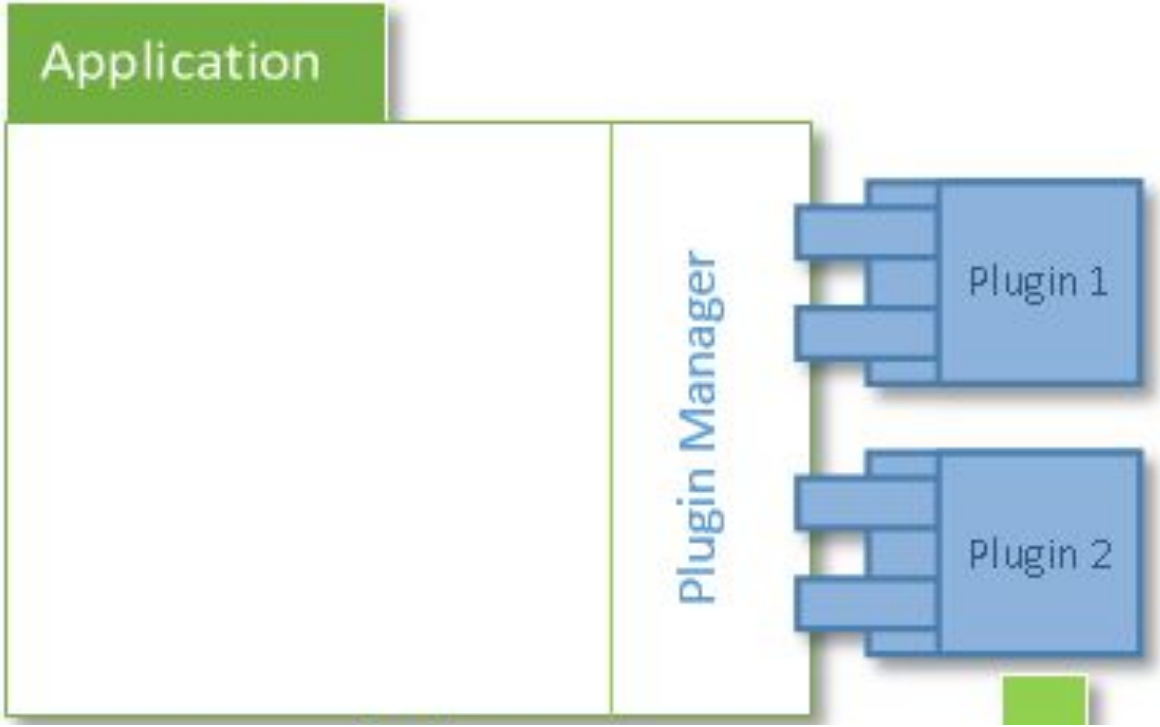
DEPENDENCY INVERSION PRINCIPLE


A
Use
API
B

High level policies should not depend on low level implementations. They should both depend on abstractions.

Design to an interface, not to an implementation!

PLUGIN ARCHITECTURE


Application
Plugin Manager
Plugin 1
Plugin 2

Dependency inversion

- Dependency types
- Compile dependency
- Runtime dependency
- Inversion
- Modularity
- **Independent deployability and developability**
- Plugin architecture
- Reuse

LISKOV SUBSTITUTION PRINCIPLE

What is a type?

Type
=
a bag of operations

```
typedef complex_number {
    double real;
    double imaginary;
};
```

Is this a type?

```
typedef complex_number {
    double real;
    double imaginary;
};
```

What about this one?

```
class ComplexNumber {
  private double real;
  private double imaginary;

  public ComplexNumber(double real, double imaginary) {
    this.real = real;
    this.imaginary = imaginary;
  }

  public double getReal() { return real; }
  public void setReal(double real) { this.real = real; }

  public double getImaginary() { return imaginary; }
  public void setImaginary(double imaginary)
  { this.imaginary = imaginary; }
}
```

```java
class ComplexNumber {
    private double real;
    private double imaginary;

    public ComplexNumber(double real, double imaginary) {
        this.real = real;
        this.imaginary = imaginary;
    }

    //getters and setters...

    public void add(ComplexNumber n) {
        //do add
    }

    public void multiply(ComplexNumber n) {
        //do multiply
    }
}
```

```
typedef complex_number {
    double real;
    double imaginary;
};
```

```
typedef described_complex_number {
    double real;
    double imaginary;
    char* description
};
```

Liskov's definition

"If for each object o1 of type T there is an object o2 of type S such that for all programs P defined in terms of T, the behavior of P is unchanged when o1 is substituted for o2 then S is a subtype of T."

Liskov, Barbara (May 1988). *Data Abstraction and Hierarchy*.

Classic example

```java
class Rectangle
{
   protected double width;
   protected double height;

   public double getWidth() { return width; }
   public void setWidth(double width)
   { this.width = width; }

   public double getHeight() { return height; }
   public void setHeight(double height)
   { this.height = height; }

   public double area() {
        return height * width;
   }
   public double perimeter() {
        return 2 * (width + height);
   }
}
```

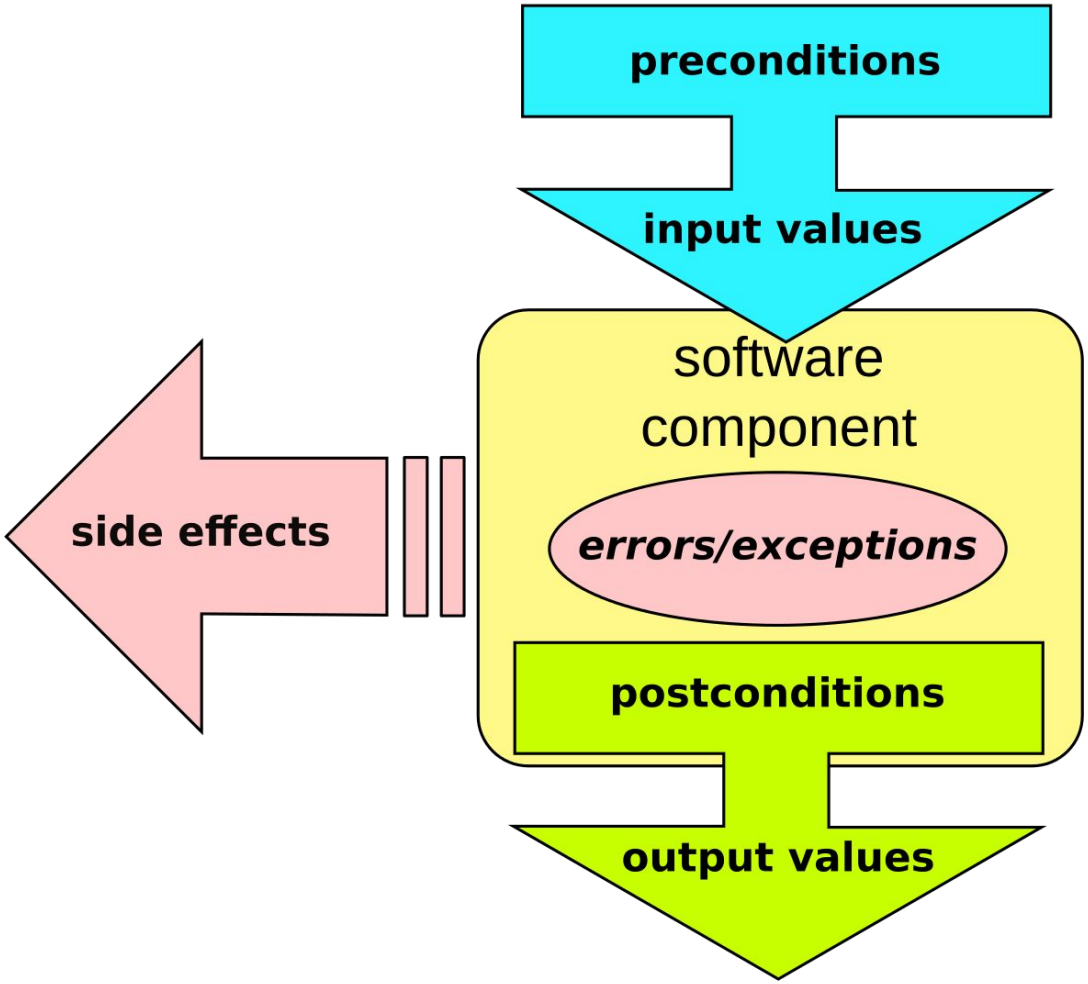
```java
class Square extends Rectangle
{

    public void setWidth(double width)
    {
        this.width = width;
        this.height = width;
    }

    public void setHeight(double height)
    {
        this.height = height;
        this.width = height;
    }

}
```

```java
public int someMethod(Rectangle r) {
        r.setWidth(10);
        r.setHeight(5);

        return r.area();
}
```

Problem if I pass in a Square???

```
public int someMethod(Rectangle r) {
        r.setWidth(10);
        r.setHeight(5);

        return r.area();
}
```

DESIGN BY CONTRACT


preconditions
input values
software
component
errors/exceptions
postconditions
output values
side effects

Liskov Substitution Principle

- What is a type?
- What is a subtype?
- Classes and subclasses
- Substitution
- Liskov's definition
- Design by contract
- Classic example (rectangle, square)
- **is about semantics/behaviour**

INTERFACE SEGREGATION PRINCIPLE

A client should not depend on interfaces (interface methods) it does not use!!!

SINGLE RESPONSIBILITY PRINCIPLE

A class, module or function should have only one responsibility!

But what is a responsibility?

Responsibility

=

Reason to change

Responsibility

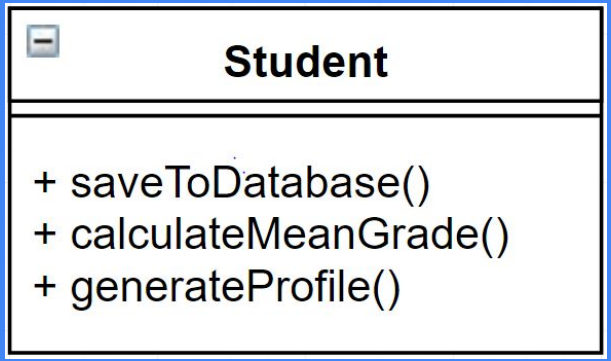
=

Reason to change

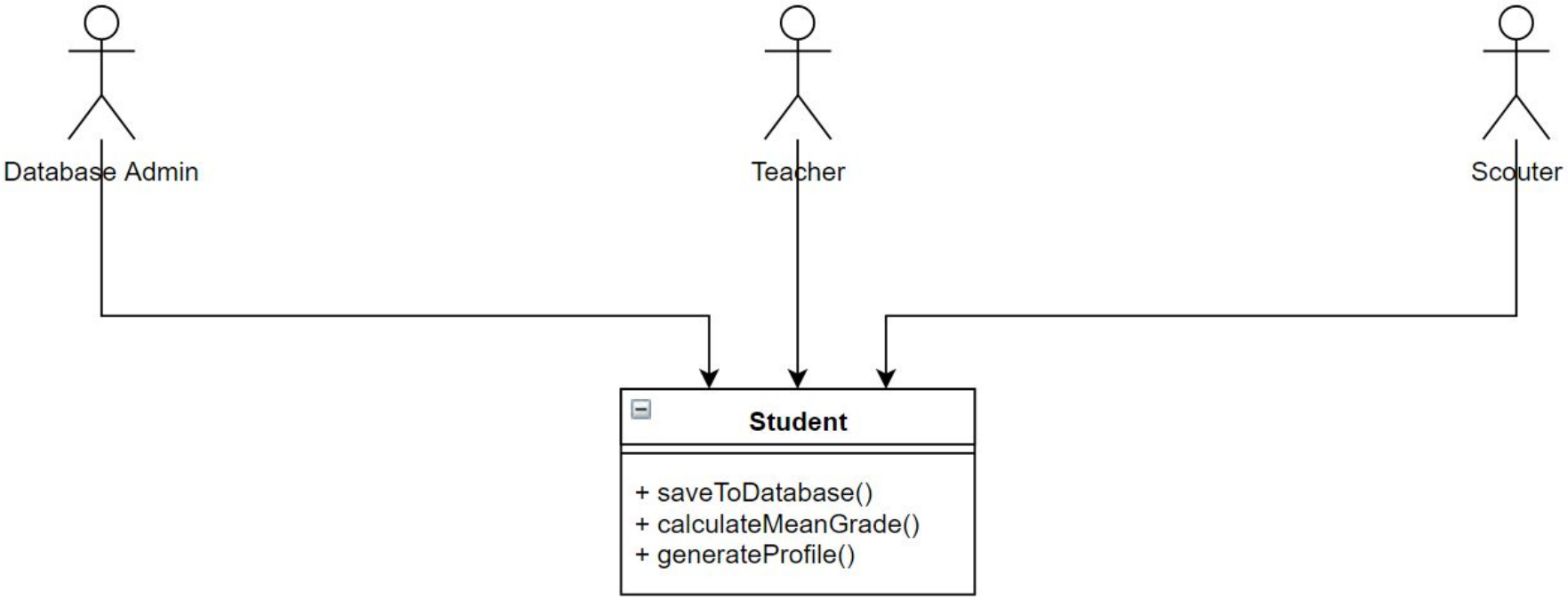
But who requests changes?

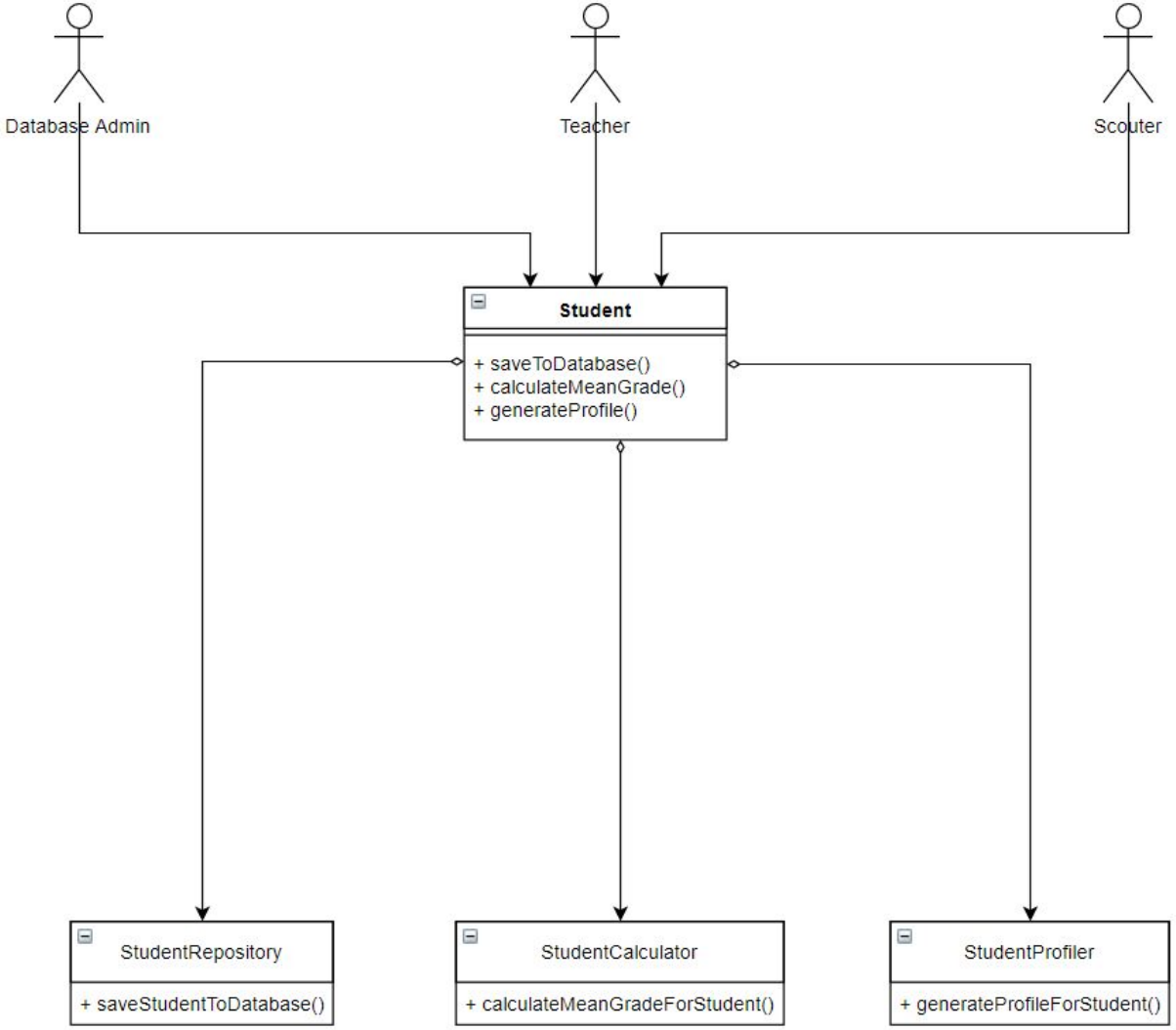
Actor

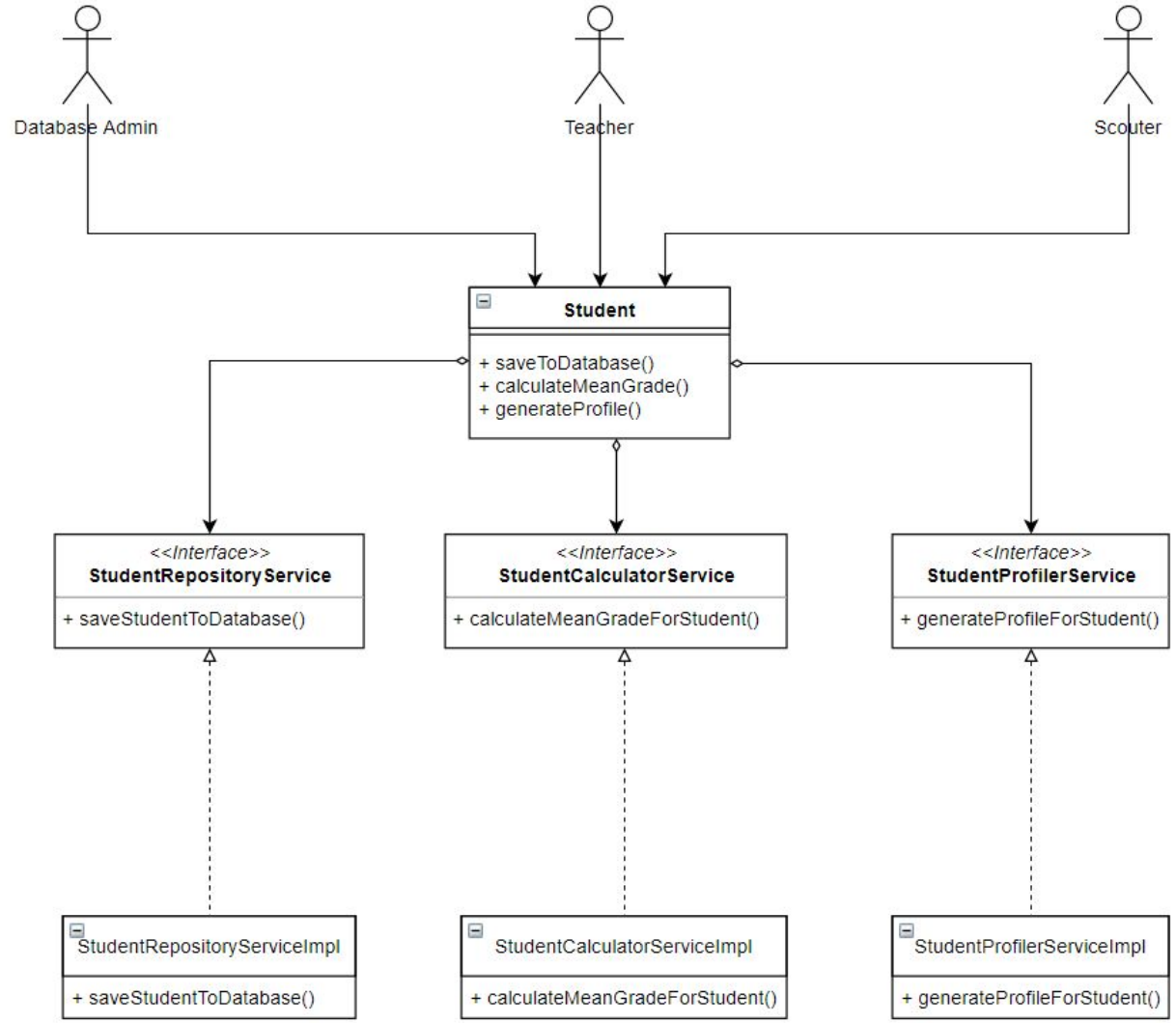
Example


Student
+ saveToDatabase()
+ calculateMeanGrade()
+ generateProfile()

Problem?


Database Admin
Teacher
Scouter
Student
+ saveToDatabase()
+ calculateMeanGrade()
+ generateProfile()


Database Admin
Teacher
Scouter
Student
+ saveToDatabase()
+ calculateMeanGrade()
+ generateProfile()
StudentRepository
+ saveStudentToDatabase()
StudentCalculator
+ calculateMeanGradeForStudent()
StudentProfiler
+ generateProfileForStudent()


Database Admin
Teacher
Scouter
Student
+ saveToDatabase()
+ calculateMeanGrade()
+ generateProfile()
<<Interface>>
StudentRepositoryService
+ saveStudentToDatabase()
<<Interface>>
StudentCalculatorService
+ calculateMeanGradeForStudent()
<<Interface>>
StudentProfilerService
+ generateProfileForStudent()
StudentRepositoryServiceImpl
+ saveStudentToDatabase()
StudentCalculatorServiceImpl
+ calculateMeanGradeForStudent()
StudentProfilerServiceImpl
+ generateProfileForStudent()

Single Responsibility

- Responsibilities $\Longleftrightarrow$ reasons to change
- It’s about actors…actually about roles
- Try to isolate responsibilities

SRP

A class should have only one reason to change.

Responsibilities

Reasons to change

Actors

Roles

OCP

A class should be open to extension and closed to modification.

Strategic closure

LSP

Subtyping is about semantics and behaviour; NOT about structure!

Inheritance

Polymorphism

Substitution

Types

Subtypes

Design by contract

ISP

A client should not depend on interfaces it does not use.

Interfaces

Dependencies

DIP

High level modules should not depend on low level details.

Interfaces

Dependencies

Reuse

Plugins